



Introdução a Técnicas de Programação

Expressões lógicas e condicionais

Prof. André Campos
DIMAp/UFRN

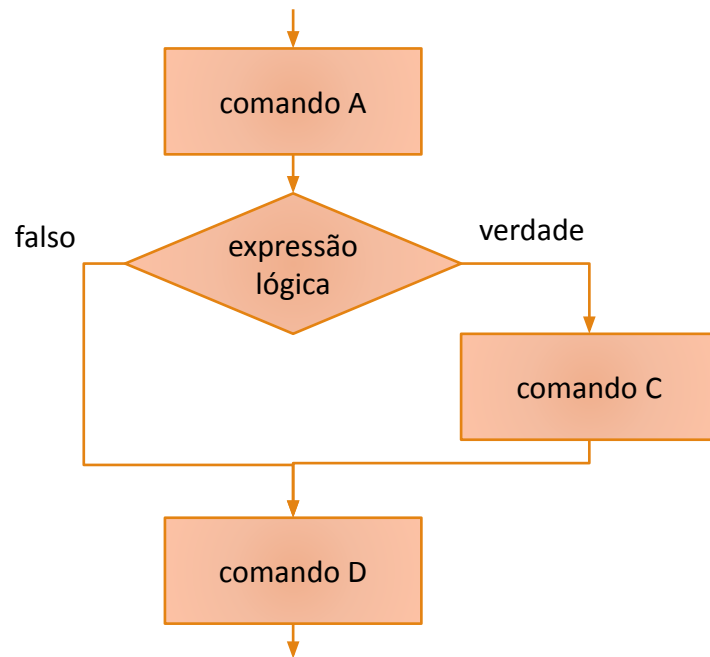
Condicional simples

Permite executar um ou mais comandos quando uma expressão lógica for **verdadeira**

```
if (expressão lógica)  
    comando a ser executado
```

Exemplo: verificar se o valor de uma variável é par.

```
if (a % 2 == 0) {  
    cout << "a é par";  
}
```

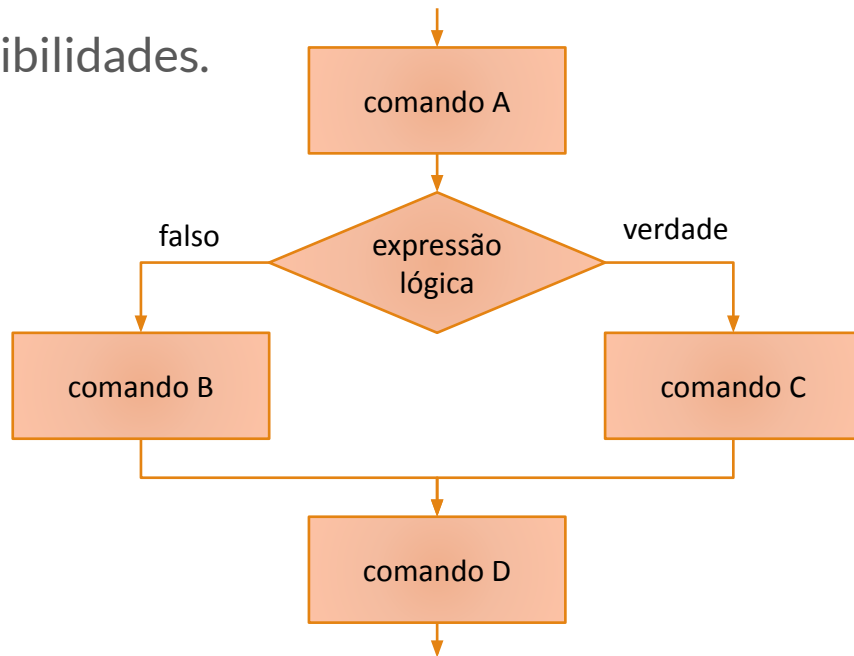


Permite executar uma dentre duas possibilidades.

```
if (expressão lógica)  
    comando a ser executado  
else  
    comando a ser executado
```

Exemplo

```
if (a % 2 == 0)  
    cout << "é par";  
else  
    cout << "é ímpar";
```





Condição aninhadas

Como o if é também um comando, ele pode se encontrar aninhado a outro if.

```
if (expressão lógica)
    comando a ser executado
else
    comando a ser executado
```

```
if (p == 0)
    cout << "C";
else
    if (r == 0)
        cout << "B";
    else
        cout << "A";
```

```
if (p == 0)
    cout << "C";
else if (r == 0)
    cout << "B";
else
    cout << "A";
```

Bloco de comandos

Definimos um bloco de comando quando precisamos executar vários comandos ao invés de um único comando.

Uso de chaves { }

```
if (expressão)
{
    comando 1;
    comando 2;
    comando 3;
    comando 4;
}
```



```
if (expressão) {
    comando 1;
    comando 2;
    comando 3;
    comando 4;
}
```

Operadores de comparação

Não confundir o operador de comparação (==) com o de atribuição (=)

Expressão em C	Teste
<code>x == y</code>	x é igual a y ?
<code>x != y</code>	x é diferente de y ?
<code>x > y</code>	x é maior que y ?
<code>x >= y</code>	x é maior ou igual a y ?
<code>x < y</code>	x é menor que y ?
<code>x <= y</code>	x é menor ou igual a y ?

Operador ternário

Considere o cálculo do valor absoluto (positivo) de uma variável.

```
int abs;

if (x >= 0) {
    abs = x;
}
else {
    abs = -x;
}
```

O valor de abs depende de uma expressão condicional.

Se $x \geq 0$, então o valor de x é atribuído a abs, caso contrário o valor de $-1 * x$ é atribuído a abs.

```
resultado = teste ? expressão1 : expressão2;
```

Podemos simplificar usando o operador ternário "?"

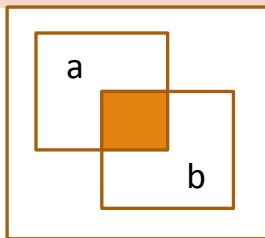
```
int abs = x >= 0 ? x : -x;
```

O resultado será expressão1 se teste for verdadeiro ou expressão2 se for falso.

Conectores lógicos

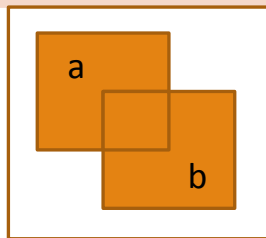
Combinam expressões lógicas

Operação	Op.	Para que a expressão resultante seja verdadeira...	Exemplo
Conjunção	& &	Ambas as expressões devem ser verdadeiras	<code>0 <= a and a <= 10</code>
Disjunção		Pelo menos uma das expressões deve ser verdadeira	<code>a <= 0 or 10 <= a</code>
Negação	!	A expressão deve ser falsa	<code>not (0 <= a and a <= 10)</code>



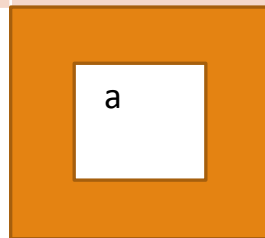
Conjunção (E)

`a & & b`



Disjunção (OU)

`a | | b`



Negação (NÃO)

`! a`



Precedência dos operadores

1. operadores unários: $-$, $!$
2. operadores binários: $*$, $/$, $\%$
3. operadores binários: $+$, $-$
4. operadores binários: $>$, $\geq$, $<$, $\leq$
5. operadores binários: $==$, $!=$
6. operador lógico: $\&\&$
7. operador lógico: $||$

Tipo booleano e expressões lógicas

Expressões lógicas, que podem assumir apenas dois valores (verdade e falso), podem ser armazenadas em variáveis booleanas.

```
bool exp = x % 2 == 1;
if (exp) {
    cout << "x é ímpar";
}
else {
    cout << "x é par";
}
```



```
if (x % 2 == 1) {
    cout << "x é ímpar";
}
else {
    cout << "x é par";
}
```

```
bool x = 5 > 2;
```



```
int x = true;
```



Casos

Mudança do fluxo de execução através de possíveis casos

Estrutura útil quando temos um conjunto fixo e limitado de opções

```
switch (expressão) {  
    case VALOR_1:  
        // comandos  
        break;  
    case VALOR_2:  
        // comandos  
        break;  
    case VALOR_3:  
        // comandos  
        break;  
    default:  
        // comandos  
}
```